DAY 03 VISUAL STUDY GUIDE • PHASE 0: FOUNDATIONS

Day 03 — CSS Fundamentals and Flexbox

Complete Revision Notes with Definitions, Code Examples, Visual Diagrams, and Practice Questions

Student: Frontend Engineering Transition
Track: React + Next.js Preparation
Milestone: Pre-CSS Grid Mastery

Visual Revision Blueprint & Milestone Objective

Welcome to your comprehensive visual notebook for **Day 03**. This guide uses color-coded diagrams, nested box schemas, coordinate axes, and concrete code demonstrations to cement your understanding of CSS before advancing to Day 04 (CSS Grid). Every concept is presented with its formal definition, purpose, runnable syntax, visual illustration, and common pitfalls to avoid.

Master Table of Contents

Part I: CSS Core, Selectors & Box Geometry

- **Chapter 1:** Introduction to CSS & Linking Pipeline
- **Chapter 2:** CSS Selectors (Element, Class, ID) & Specificity
- **Chapter 3:** The CSS Box Model & Geometry Calculations
- **Chapter 4:** Margin and Padding Clockwise Shorthand
- **Chapter 5:** Typography Metrics & Readability Controls
- **Chapter 6:** Colors, Backgrounds & Color Spaces
- **Chapter 7:** Borders, Border Radius & Geometric Shapes
- **Chapter 8:** The HTML `<div>` Container & Hierarchy

Part II: Display, Positioning & Flexbox Layouts

- **Chapter 9:** Display Modes: Block, Inline & Inline-Block
- **Chapter 10:** CSS Positioning: All 5 Schemes Visualized
- **Chapter 11:** Flexbox Fundamentals: Axes & Flow Directions
- **Chapter 12:** Main Axis Alignment vs Cross Axis Alignment
- **Chapter 13:** Flexbox Gap, Wrapping & Multi-Line Distribution
- **Chapter 14:** Complete Hands-on Project: Skills Card
- **Chapter 15:** Complete Day 03 Quick Revision Table
- **Chapter 16:** 15-Question Visual Revision Quiz & Checklist

Chapter 1 — Introduction to CSS

CSS stands for **Cascading Style Sheets**. It is a declarative stylesheet language used to control the visual presentation, color palette, typography, spacing, and geometric layout of HTML elements on a webpage.

HTML: The Structural Skeleton

- Defines the computational structure and hierarchy.
- Creates raw content: headings, paragraphs, buttons, lists.
- Provides meaning and accessibility landmarks.

CSS: The Visual Presentation

- Defines how HTML elements are painted on screen.
- Controls color palettes, fonts, line spacing, and borders.
- Governs responsive multi-device layout architecture.

External Stylesheet Linking Pipeline

```html
<!-- HTML Document (index.html) -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My Page</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Hello World</h1>
  <p>Learning CSS</p>
</body>
</html>
```

```css
/* External Stylesheet (style.css) */
h1 {
  color: purple;
  font-size: 32px;
}

p {
  color: gray;
}
```

`<link>` : Void tag connecting external resources.
`rel="stylesheet"` : Defines the relationship as a CSS style document.
`href="style.css"` : Path to the external CSS stylesheet.

VISUAL ARCHITECTURE: BROWSER RENDERING PIPELINE

1. HTML File DOM Structure + **2. External CSS** Parsed Rules (CSSOM) → **3. Browser Engine** Calculates Render Tree → **4. Styled Webpage** Painted User Interface

Chapter 2 — CSS Selectors

A **CSS selector** identifies the HTML elements targeted by a styling rule. Choosing the correct selector ensures predictable styling without unintended side effects.

1. ELEMENT SELECTOR

Syntax: `p { ... }`

Targets elements using their native HTML tag name. Applies globally across the entire page.

Targets: `<p>`, `<p>`, etc.

2. CLASS SELECTOR

Syntax: `.description { ... }`

Starts with a period (`.`). Reusable across multiple elements on the same page.

Targets: Any element with `class="description"`

3. ID SELECTOR

Syntax: `#title { ... }`

Starts with a hash (`#`). Targets a single unique element per document.

Targets: Single element with `id="title"`

Selector Type	Syntax Prefix	Targeted DOM Elements	Recommended Practical Usage
Element	None (Tag name: `p`)	All paragraph elements globally	Baseline typography and default element resets
Class	Dot (`.description`)	Any element carrying that specific class	**Primary UI styling:** Reusable cards, buttons, badges
ID	Hash (`#title`)	The single element with that specific ID	Unique page landmarks, form labels, fragment anchors

Chapter 3 — CSS Box Model

The **CSS Box Model** dictates how every HTML element is calculated as a rectangular bounding box. The box consists of four concentric spatial layers:

VISUAL DIAGRAM: THE 4 CONCENTRIC LAYERS OF THE CSS BOX MODEL

MARGIN (Light Yellow: Outer empty separation between elements) 10px

BORDER (Orange: Physical boundary around padding & content) 2px solid black

PADDING (Light Green: Inner space between border & content) 20px

CONTENT AREA (White: Text, images, and child DOM nodes)

width: 300px; height: auto;

```
.card {
  width: 300px;
  padding: 20px;
  border: 2px solid black;
  margin: 10px;
}
```

Total Width Calculation (Standard Content-Box):

```
Content (300px) + Padding L/R (40px) +
Border L/R (4px) = 344px Total
```

Margin Notice: The 10px margin sits completely outside the element's border. It pushes neighboring elements away but does not change the element's internal 344px box footprint.

> *Note: Adding `box-sizing: border-box` absorbs padding and borders inside the declared 300px width.*

Chapter 4 — Margin and Padding Shorthand

CSS shorthand lets you specify spatial boundaries concisely. Multi-value shorthand properties follow a strict **clockwise order (Top → Right → Bottom → Left)**:

CLOCKWISE SHORTHAND COMPASS (TRBL)

1. TOP (10px)
4. LEFT (40px) ↻ Clockwise 2. RIGHT (20px)
3. BOTTOM (30px)

Code: `padding: 10px 20px 30px 40px;`

Values	Shorthand Rule	Expanded Equivalent
1 Value	`padding: 10px;`	All 4 sides get 10px
2 Values	`margin: 10px 20px;`	10px Vertical (T/B), 20px Horizontal (L/R)
3 Values	`padding: 10px 20px 30px;`	10px Top, 20px L/R, 30px Bottom
4 Values	`margin: 10px 20px 30px 40px;`	10px Top, 20px Right, 30px Bottom, 40px Left

Chapter 5 — Typography

Typography controls the readability and visual hierarchy of text content on the page:

```
h1 {
  font-family: Arial, sans-serif;
  font-size: 32px;
  font-weight: bold;
}

p {
  font-size: 16px;
  line-height: 1.5;
  text-align: center;
}
```

`font-family` : Prioritized typeface list with a generic fallback (`sans-serif`).
`line-height: 1.5;` : Vertical spacing multiplier (1.5× font size).
`text-align` : Horizontal alignment (`left` , `center` , `right`).

RENDERED BROWSER OUTPUT PREVIEW

Large Centered Heading

This paragraph is styled with clean **Arial typography**, an expanded readable line-height of 1.5, and centered horizontal alignment.

Chapter 6 — Colors and Backgrounds

The `color` property styles foreground typography, whereas `background-color` fills the element's container surface:

Named Color	HEX Notation	RGB Functional
`color: purple;`	`color: #2563eb;`	`rgb(128, 0, 128);`

Syntax Watchout:
Never write `background-color: light blue;` with a space! Spaces separate distinct property values in CSS syntax. Always use the continuous token `lightblue` or its HEX equivalent (`#add8e6`).

```
body {
  background-color: #f5f5f5;
}

h1 {
  color: purple;
}

button {
  background-color: #2563eb;
  color: white;
}
```

Chapter 7 — Border and Border Radius

A **border** frames an element's content and padding, while `border-radius` curves the outer corners:

SQUARE BORDER

`border: 2px solid black;`
`border-radius: 0px;`
Sharp, 90-degree angular box corners.

ROUNDED BORDER

`border: 2px solid #2563eb;`
`border-radius: 12px;`
Smooth, modern curved corners.

- **Border Shorthand:** `border: 2px solid black;` sets width (2px), style (solid), and color (black) in one line.
- **Resetting Native Borders:** `border: none;` strips away default bevel styling from native HTML `<button>` elements.

Chapter 8 — Div and Grouping Elements

A `<div>` (division) is an unopinionated, generic block-level container used to bundle related child nodes together for unified CSS styling and layout manipulation:

VISUAL DIAGRAM: PARENT-CHILD ENCLOSURE STRUCTURE

PARENT CONTAINER:

<h2>Java Developer</h2> (First Child Node)

Learning CSS (Second Child Node)

Element	Semantic Meaning	Primary Recommended Usage
`<div>`	Non-semantic (Generic)	Purely layout and visual styling wrappers with classes
`<section>`	Semantic Thematic Group	Content sections typically anchored by an `<h2>` heading
`<header>`	Semantic Landmark	Introductory title banner, logo, or navigation masthead
`<main>`	Semantic Core Container	Wraps the single, non-repeating primary content of the page

Chapter 9 — Display Properties

The `display` property controls how an element interacts with surrounding content in the document flow:

1. DISPLAY: BLOCK;

Takes full width; stacks on new lines.

Box 1

Box 2

Box 3

2. DISPLAY: INLINE;

Flows with text; ignores width & height.

Text flows Box 1 and then Box 2 inside one line.

3. DISPLAY: INLINE-BLOCK;

Side-by-side while respecting dimensions.

50×35 50×35

Property	Forces New Line?	Width & Height Supported?	Typical HTML Elements
`block`	Yes	Yes (Fully respected)	`<div>`, `<p>`, `<h1>` – `<h6>`, `<section>`
`inline`	No	No (Ignored by the browser)	`<span>`, `<a>`, `<strong>`, `<em>`
`inline-block`	No	Yes (Fully respected)	`<button>`, `<img>`, `<input>`

Chapter 10 — CSS Positioning Schemes

The `position` property overrides default document layout flow, allowing elements to be precisely offset using `top`, `right`, `bottom`, and `left`:

10.1 STATIC POSITIONING

Default browser flow. Offset coordinates (`top`, `left`) have zero effect.

Box 1 → Box 2 → Box 3 (Normal Flow)

10.2 RELATIVE POSITIONING

Shifts relative to itself. *Original space remains reserved in the document!*

Moved Box

10.3 ABSOLUTE POSITIONING (CARD WITH NOTIFICATION BADGE)

Removed completely from normal flow. Positions relative to the nearest positioned ancestor (`position: relative`).

```
.card {
  position: relative; /* Containing Block */
  width: 200px;
  background: white;
  border: 2px solid black;
}

.badge {
  position: absolute;
  top: 8px;
  right: 8px;
  background: gold;
}
```

Product Card NEW

Parent has `position: relative`.

Badge has `position: absolute`.

10.4 FIXED POSITIONING

Anchors to the **browser viewport**; remains fixed when scrolling.

Page content scrolling...

Fixed Bottom Bar (Stays on screen)

10.5 STICKY POSITIONING

Flows normally until scroll threshold (`top: 0`), then pins in place.

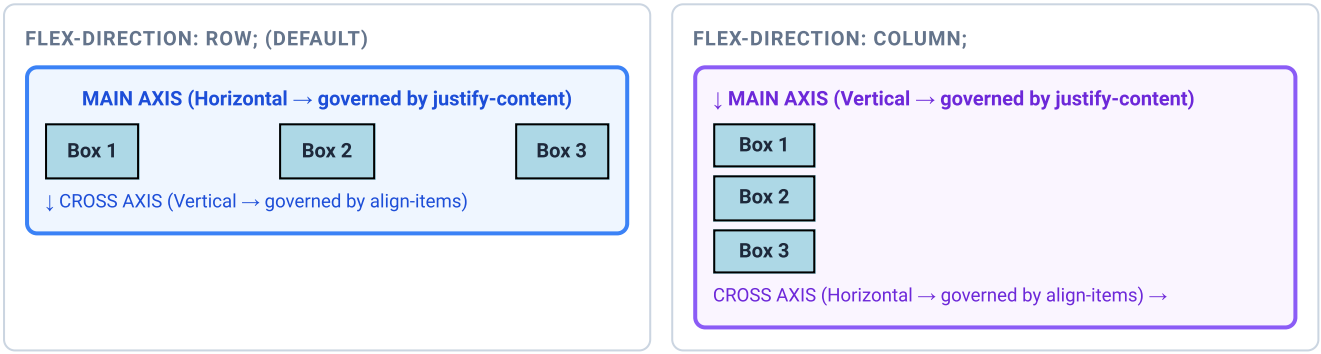
Sticky Header (Pins to top on scroll)

Section content scrolling under header...

Position Value	In Normal Document Flow?	Positional Reference Origin	Typical Real-World Usage
`static`	Yes	Normal flow; offsets ignored	Default document layout
`relative`	Yes (Original space reserved)	Its own natural unshifted position	Slight offsets; anchor parent for absolute children
`absolute`	No (Removed from flow)	Nearest positioned ancestor	Notification badges, close icons, floating tooltips
`fixed`	No (Removed from flow)	The browser **viewport window**	Fixed navigation bars, floating action buttons
`sticky`	Hybrid (Flows until threshold)	Nearest scrolling container	Sticky table headers, pinned navigation bars

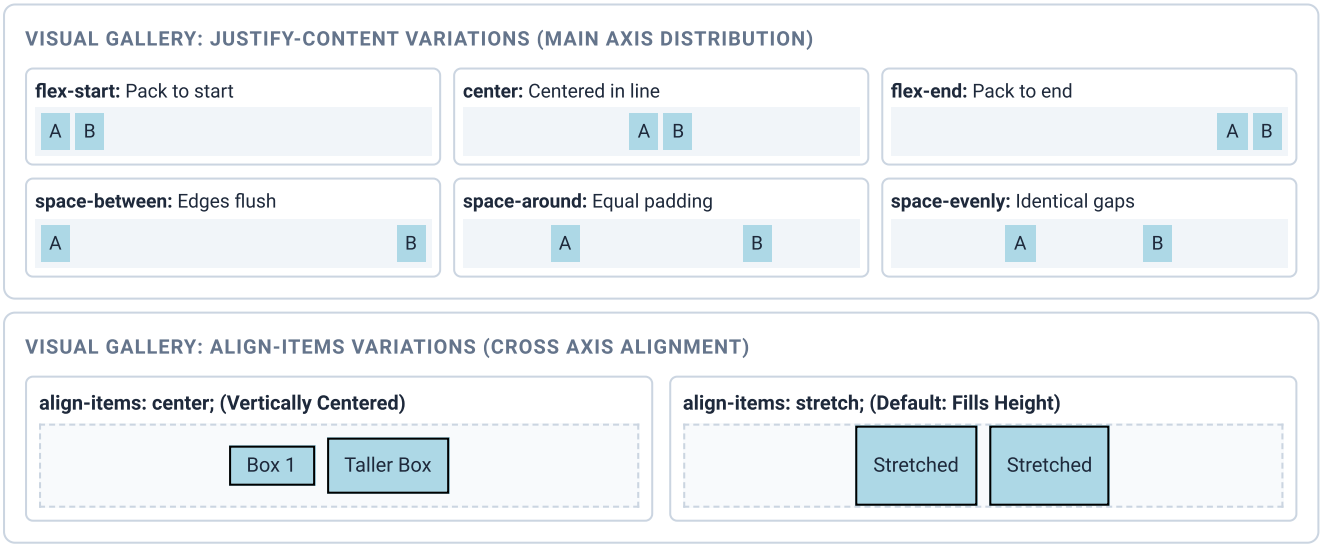
Chapter 11 — Flexbox Fundamentals

The **Flexible Box Layout (Flexbox)** module provides a 1-dimensional layout model designed to distribute space and align items along either a horizontal row or a vertical column.



Chapter 12 — Justify-Content vs Align-Items

`justify-content` controls alignment along the **Main Axis**, while `align-items` controls alignment along the **Cross Axis** within each flex line:



Chapter 13 — Flexbox Gap, Wrapping & Align-Content

The `gap` property creates gutter spacing between items, while `flex-wrap` enables multi-line wrapping:

FLEX-WRAP: NOWRAP; (SQUISHED SINGLE LINE)

1 2 3 4 5 6

FLEX-WRAP: WRAP; WITH GAP: 8PX; (MULTI-LINE)

Box 1 Box 2 Box 3 Box 4

The Difference: `align-items` **vs** `align-content` **:**
- `align-items` : Aligns individual items along the cross axis *within each flex line*.
- `align-content` : Distributes *entire flex lines relative to each other* along the cross axis. *Only takes effect when* `flex-wrap: wrap;` *is active and extra vertical space exists in the container!*

Chapter 14 — Complete Flexbox Project: Skills Card

The following mini-project brings together container styling, card grouping, border radius, flex wrapping, and gap spacing:

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My Skills</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="card">
    <h1>My Skills</h1>
    <div class="container">
      <div class="box">Java</div>
      <div class="box">Spring Boot</div>
      <div class="box">MySQL</div>
      <div class="box">HTML</div>
      <div class="box">CSS</div>
      <div class="box">Git</div>
    </div>
  </div>
</body>
</html>
```

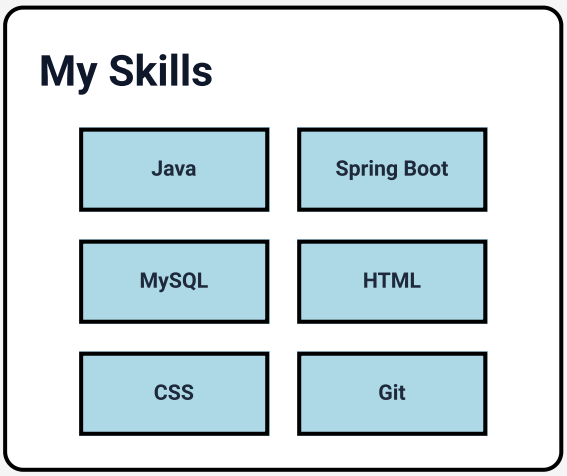
```css
body {
  background-color: #f5f5f5;
  font-family: Arial, sans-serif;
}

.card {
  width: 300px;
  background-color: white;
  padding: 20px;
  border: 2px solid black;
  border-radius: 10px;
}

.container {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
}

.box {
  width: 100px;
  height: 50px;
  background-color: lightblue;
  border: 2px solid black;
  text-align: center;
}
```

RENDERED VISUAL SIMULATION (300PX WHITE CARD WITH WRAPPING FLEX ITEMS)



Chapter 15 — Complete Day 03 Revision Table

Topic / Concept	Core Definition & Purpose	Key CSS Property & Example Syntax
CSS	Cascading Style Sheets; controls appearance and layout	Linked via `<link rel="stylesheet" href="style.css">`
Element Selector	Selects all matching HTML tag elements globally	`p { color: blue; }`
Class Selector	Reusable selector targeting elements by class attribute	`.description { color: green; }`
ID Selector	Unique selector targeting the single element with matching ID	`#title { color: purple; }`
Box Model	4 concentric layers: Content, Padding, Border, Margin	`width`, `padding`, `border`, `margin`
Margin	Transparent spacing outside the element's border	`margin: 10px 20px;`
Padding	Internal breathing room between content and border	`padding: 20px;`
Typography	Controls typeface, character sizing, weight, and line height	`font-family`, `font-size`, `line-height: 1.5;`
Colors	Foreground text color vs surface background tint	`color`, `background-color`
Border Radius	Curves rectangular corners into rounded edges or circles	`border-radius: 10px;`
Div Container	Generic non-semantic block container for grouping elements	`<div class="card">`
Display Modes	Governs document flow behavior (`block`, `inline`, `inline-block`)	`display: block | inline | inline-block;`
Positioning	Overrides document flow with coordinate offsets	`position: static | relative | absolute | fixed | sticky;`
Flexbox	1-dimensional layout system along main and cross axes	`display: flex;`
Flex Direction	Sets orientation of the main axis (`row` or `column`)	`flex-direction: row | column;`
Justify Content	Distributes items along the **Main Axis**	`justify-content: center | space-between;`
Align Items	Aligns items along the **Cross Axis** within a line	`align-items: center | stretch;`
Flex Gap	Generates consistent gutter spacing between flex items	`gap: 20px;`
Flex Wrap	Allows overflowing items to break onto multiple rows	`flex-wrap: wrap;`
Align Content	Distributes multiple flex lines along the cross axis	`align-content: center;`

Chapter 16 — Visual Revision Quiz (15 Questions)

1. **What does CSS stand for?** (A) Creative Style Sheets (B) Cascading Style Sheets (C) Computer Style Sheets (D) Colorful System Styles
   Answer: **B** — Cascading Style Sheets.
2. **Which symbol represents a class selector in CSS?** (A) `#` (B) `.` (C) `&` (D) `@`
   Answer: **B** — A dot/period precedes all class selectors (`.card`).
3. **Which symbol represents an ID selector?** (A) `.` (B) `#` (C) `!` (D) `*`
   Answer: **B** — The hash symbol precedes unique ID selectors (`#title`).
4. **In the CSS Box Model, which layer is inside the border and surrounds the content?** (A) Margin (B) Outline (C) Padding (D) Gap
   Answer: **C** — Padding lives inside the border.

5. **Under standard content-box sizing: width is 300px, padding is 20px on each side, border is 2px. What is the total rendered width?** (A) 300px (B) 324px (C) 344px (D) 364px
   Answer: **C** — 300 + 20 + 20 + 2 + 2 = 344px.
6. **What order does `padding: 10px 20px 30px 40px;` follow?** (A) Left, Right, Top, Bottom (B) Clockwise: Top, Right, Bottom, Left (C) Top, Bottom, Left, Right (D) Counter-clockwise
   Answer: **B** — Always follows clockwise order: Top, Right, Bottom, Left.
7. **Which property changes the color of typography text?** (A) `font-color` (B) `text-color` (C) `color` (D) `background-color`
   Answer: **C** — `color` controls text foreground color.
8. **Which display mode allows elements to sit side-by-side while respecting explicit width and height?** (A) block (B) inline (C) inline-block (D) none
   Answer: **C** — `inline-block` combines inline flow with block dimensions.
9. **What happens to an element when `position: relative; top: 20px;` is applied?** (A) It is removed from normal flow (B) It shifts 20px down from its original position while retaining its original layout space (C) It anchors to viewport (D) It turns into a flexbox
   Answer: **B** — It shifts visually, but its original space remains reserved in the document.
10. **Where does an absolutely positioned element look for its reference origin?** (A) The body element only (B) The nearest positioned ancestor (an element with non-static position) (C) The browser window only (D) Its direct sibling
    Answer: **B** — It anchors to the nearest positioned ancestor (typically `position: relative`).
11. **Which position scheme stays locked at the exact same coordinates on screen during page scrolling?** (A) relative (B) absolute (C) fixed (D) static
    Answer: **C** — `fixed` positions relative to the browser viewport.
12. **In a row-based flex container (`flex-direction: row`), which axis is horizontal?** (A) Cross axis (B) Main axis (C) Z-axis (D) Vertical axis
    Answer: **B** — In row mode, the Main Axis is horizontal and controlled by `justify-content`.
13. **Which flex property distributes items along the main axis?** (A) `align-items` (B) `justify-content` (C) `align-content` (D) `flex-grow`
    Answer: **B** — `justify-content` controls main-axis spacing.
14. **Which flex property aligns items along the cross axis within each flex line?** (A) `justify-content` (B) `align-items` (C) `gap` (D) `flex-direction`
    Answer: **B** — `align-items` aligns items within their line.
15. **When does `align-content` take effect?** (A) In single-line containers (B) Only when `flex-wrap: wrap;` is active and multiple flex lines exist with extra vertical space (C) On inline text spans (D) Always
    Answer: **B** — It only operates on multi-line flex containers.

Day 03 Revision Checklist & Next Milestone

- [ ] I understand CSS and how to link an external stylesheet
- [ ] I can use element, class, and ID selectors accurately
- [ ] I understand the 4 layers of the CSS Box Model
- [ ] I understand margin vs padding and clockwise shorthand
- [ ] I understand typography properties and line-height
- [ ] I can use named colors, HEX, and RGB notation
- [ ] I can style borders, border-radius, and rounded corners
- [ ] I understand `<div>` and parent-child nesting relationships
- [ ] I understand block, inline, and inline-block display modes
- [ ] I understand static, relative, absolute, fixed, and sticky
- [ ] I understand Flexbox containers and flex items
- [ ] I understand row and column flow directions
- [ ] I understand main axis vs cross axis orientation
- [ ] I understand justify-content and align-items
- [ ] I understand gap, flex-wrap, and align-content
- [ ] I can build a responsive skills card layout with Flexbox

Day 03 Complete — Ready for Day 04: CSS Grid

You have mastered 1-dimensional layouts with Flexbox and the foundational geometry of the CSS Box Model. On **Day 04**, you will expand into 2-dimensional layouts using **CSS Grid** (grid areas, template columns, auto-fit/minmax, and responsive dashboard layouts).